ISYS2120 Sem2, 2025

Assignment 3 (Concepts)

SID: 540243105

Question A

$$\pi_{STitle}(\sigma_{\text{PUser}='Richard' \wedge SDuration<180}((PlaylistSong \bowtie Playlist) \bowtie Song))$$

$$\pi_{STitle}((\sigma_{\text{PUser}='Richard'}(Playlist) \bowtie PlaylistSong) \bowtie \sigma_{SDuration<180}(Song))$$

Question B

B(i)

The functional dependency **Team, GDate → Score, Location** means that:

For each team on a given date, the score and location of the game are uniquely determined. And a team can only play 1 game per date, which has a specific score and location.

example of data:

SportsInfo('Sally Snape', 'Sydney', 'swan', '30-03-2024', 56,'Olympic Park', 15000)

SportsInfo('Sally Snape', 'Sydney', 'swan', '30-03-2024', 70,'Olympic Park', 15000)

This violates the dependency because the same team (Sydney) on the same date (30-03-2024) has two different scores.

B(ii)

$$\begin{aligned}(Team, GDate)^+ &= \{Team, GDate\} \\ &= \{Team, Mascot, GDate\} \text{ as } Team \rightarrow Mascot \\ &= \{Team, Mascot, GDate, Score, Location\} \text{ as } Team, GDate \rightarrow Score, Location\end{aligned}$$

B(iii)

A relation is in **BCNF** if for every FD **X → Y**, **X** is a superkey or $Y \subseteq X$.

But for fd (Team, GDate → Score, Location), it follows from B(ii) that (Team, GDate)+ miss Player and Fee. Therefore, (Team, GDate) is not a superkey and also not second condition.

B(iv)

FD: (Team, GDate → Score, Location)

R2 is SportsInfo columns, leaving out rhs of the fd; R1 is combination of lhs and rhs of fd

X ∩ Y is {Team, GDate}, and (Team, GDate)+ contains all the columns of R1

So decomposing SportsInfo into R1 and R2, is a lossless-join decomposition

$$R_1(X) = R_1(Team, GDate, Score, Location)$$
$$\text{FD: } Team, GDate \rightarrow Score, Location$$
$$\text{Primary Key: } (Team, GDate)$$

$$R_2(Y) = R_2(Player, Team, Mascot, GDate, Fee)$$
$$\text{FDs: } Player \rightarrow Team; Team \rightarrow Mascot; Player, GDate \rightarrow Fee$$
$$\text{Primary Key: } (Player, GDate)$$

Question C

C(i)

Since the only existing index is the clustered index(Primary Key) on (SId, Genre), the database can efficiently search for all rows where SId=17, because SId is the first column of the clustered index key.

However, within those rows, it must check each tuple to see if PopularityScore<4. This is because there is no index on PopularityScore.

So the steps are:

1. Use the clustered index to find all records with SId=17.
2. For each of those rows, check whether PopularityScore<4.
3. Output the corresponding Genre values that satisfy both conditions.

C(ii)

```
CREATE INDEX idx_GenreInfo
ON GenreInfo(SId, PopularityScore);
```

1. The index is ordered by SId first, then PopularityScore
2. For SId=17, the SQL can directly access only the relevant index entries
3. Then it can perform a range scan for PopularityScore<4 within those entries
4. Finally, it returns the Genre column from the index

Question D

D(i)

Goal

Normal users (without isAdmin) cannot modify or delete other users' data — this upholds data integrity.

Mechanism

1. **Role-based access control**:
   - On login, the app fetches the user's role from the UserRoles table and stores it in the session
   - Routes that modify data must check this flag on the server side
   - Only admins can perform full update/delete operations, while normal users can only modify their own record
2. **Server-side authorization**:
   - Authorization must be enforced in the processing layer (Flask server), never on the client.
   - This ensures the application, not the browser, decides what SQL the DB executes.
3. **Parameterized SQL queries**:
   - All write operations should use DB-API parameter passing.
   - No string concatenation or f-strings, which can cause SQL injection.
4. **Secure sessions and error handling**:
   - Disable debug=True in production.
   - Do not display internal DB errors to users — log them server-side.
   - Use a strong, random secret_key, not a hard-coded string.
5. **Least-privilege DB account**:
   - The web app should connect using a restricted DB user account with only the necessary CRUD permissions.
   - Privileges should be centrally managed using DB roles.
6. **Integrity constraints & DB guardrails**:
   - Add database-level constraints to prevent invalid writes.

- Use stored procedures to encapsulate updates for auditing.

7. **Safe output encoding (XSS protection)**: Use Jinja's default HTML escaping when rendering user input from the database to prevent persistent XSS.
8. **Minimize exposure of raw tables**: Provide restricted views for non-admins.

Weaknesses

1. Missing authorization checks: Only check login, not admin. Any logged-in user can modify or delete others.
2. Weak session secret: Hard-coded key allows attackers to forge session cookies.
3. SQL injection: In update_single_user() and delete_user(), user input is interpolated directly into SQL: f"WHERE userid={userid}". Attackers could send userid="1; DROP TABLE users;".
4. Overexposed data: Base tables are queried directly; no row/column-level filtering for normal users.
5. Missing DB constraints: No CHECK on user roles — invalid roles can be inserted or privileges escalated.
6. Information leakage: debug=True and detailed tracebacks reveal DB schema and SQL.

D(ii)

config.ini stores database connection details (host, username, password), plus possible API keys and encryption secrets. This file effectively grants full access to the production database and must be treated as confidential business data.

If leaked:

1. Integrity and confidentiality destroyed. Attackers can connect directly to the DB using the same credentials as the app.
2. Operational outage. They can read, alter, or delete sensitive data — bypassing all app-level access control.
3. They could impersonate users, inject malicious content, or take down the service.
4. Legal exposure and reputation loss. It may trigger data breach notifications, legal and compliance obligations, and reputation damage.

Protection measures

1. File & repository hygiene: Keep config.ini **outside the web root**, never share in email or chat.
2. Permission control: Restrict OS file permissions.
3. Secret management: Store credentials in environment variables or a secret manager instead of plaintext files.
4. Least privilege: Use a DB account with minimal privileges; no administrative.

5. Credential rotation: Change DB passwords and API keys periodically; revoke old ones after incidents.
6. Encryption & hashing: Store user passwords in DB as salted hashes; never plaintext.
7. Debug & patching: Disable debug mode; keep dependencies and DBMS updated patch.
8. Fail safely: If config errors occur, show a friendly message; log technical details privately.